



Proyecto 1: Visualizador de Datos con Ordenamiento para Linux

ENLACE A REPOSITORIO:

[https://github.com/scisneros29/Proyect1---Arquitectura_Computadores---S2---
2025/tree/main](https://github.com/scisneros29/Proyect1---Arquitectura_Computadores---S2---2025/tree/main)

Curso:

Arquitectura de Computadores

Profesor:

Ronny García

Estudiante:

Santiago Cisneros Hernández - 2020058823

ACLARACIÓN DEL REPOSITORIO:

Se trabajaron los pasos 1, 2 y 3 en branches separadas a la principal, con el objetivo de tener mejor ordenamiento en cada uno de los pasos. Cada una de las branch tiene commits de los cambios realizados en cada paso. El paso 4 está en la branch principal (main).

Objetivo Principal

El presente trabajo técnico consiste en un código en lenguaje ensamblador CISC x86 que sea capaz de ordenar ciertos datos de un archivo de texto, enlazarlos con un carácter de un archivo de configuración y reordenarlos en orden alfabético para mostrarlos en una consola de Linux. Más específicamente, se solicita leer parámetros de configuración desde config.ini, cargar y procesar Inventario.txt, ordenar alfabéticamente los ítems y renderizar en la terminal un gráfico de barras coloreado con códigos ANSI, mostrando también la cantidad de cada ítem.

Formato de archivos

- config.ini

caracter_barra:<utf8> (1–8 bytes, admite UTF-8)

color_barra:<decimal>

color_fondo:<decimal>

Defaults si faltan claves: caracter_barra='*', color_barra=92, color_fondo=40.

- inventario.txt

Una línea por ítem: nombre: cantidad (ej. manzanas:12).

Estructuras de datos (sección .bss)

- config_buf (2 KB) y config_len: buffer y bytes válidos del config.
- bar_bytes (8B) y bar_len: carácter de barra (UTF-8) y longitud real.
- color_barra, color_fondo: enteros con códigos ANSI.
- num_buf/num_len: buffer temporal para conversión u32→decimal.

- `inv_buf` (4 KB) y `inv_len`: contenido de inventario.
- `inv_count`: cantidad de ítems parseados.
- Arrays paralelos (SoA) para `MAX_ITEMS` (=128):
 - `inv_name_ptrs` (qword): punteros a nombres dentro de `inv_buf`.
 - `inv_name_lens` (dword): longitudes de nombres.
 - `inv_qtys` (dword): cantidades.
- `inv_max_qty`: máxima cantidad (para escalar la barra).

Diseño SoA: facilita ordenar por nombre y mantener sincronizados los campos.

Flujo general

Defaults: inicializa barra y colores por omisión.

• *Paso 1*

El objetivo de esta etapa fue leer y procesar parámetros iniciales desde un archivo externo denominado `config.ini`, el cual define las propiedades de la barra que será utilizada en la representación gráfica del inventario.

El archivo se abre en modo solo lectura (`O_RDONLY`) y se carga en un buffer intermedio (`config_buf`) mediante la syscall `read`. En caso de error durante la apertura o lectura, el programa aborta mostrando el mensaje correspondiente y retorna con un código de salida predefinido.

Parámetros admitidos en `config.ini`:

- `caracter_barra`: símbolo utilizado para dibujar cada segmento de la barra (soporta hasta 8 bytes, permitiendo caracteres ASCII o UTF-8).
- `color_barra`: código decimal que representa el color de la barra en secuencias ANSI.
- `color_fondo`: código decimal que representa el color de fondo de la barra.

Procedimiento de parseo:

- Valores por defecto:

Antes de leer el archivo, se definen valores iniciales:

- `caracter_barra = '*'`
- `color_barra = 92` (verde brillante)
- `color_fondo = 40` (fondo negro)

Estos se mantienen en caso de que alguna clave no aparezca en el archivo.

- Búsqueda de claves en el buffer:

Se recorre secuencialmente el contenido cargado en `config_buf`.

Para cada clave (`caracter_barra:`, `color_barra:`, `color_fondo:`) se realiza una comparación byte a byte contra el texto.

- Extracción de valores:

Para `caracter_barra:` se salta cualquier espacio posterior y se guarda el primer token de la línea en `bar_bytes` junto con su longitud (`bar_len`).

Para `color_barra:` y `color_fondo:` se realiza un parseo numérico carácter por carácter hasta conformar el valor decimal.

- Almacenamiento de resultados:

Los valores extraídos reemplazan a los definidos por defecto.

Si alguna clave no se encuentra, se conservan los parámetros iniciales.

Una vez completada la lectura, el sistema imprime en consola los parámetros detectados, confirmando la configuración efectiva que será usada en los pasos siguientes.

- *Paso 2*

En esta etapa se incorporó la funcionalidad de lectura del archivo externo `inventario.txt`, cuyo contenido describe una lista de ítems con su nombre y cantidad. El propósito fue cargar los datos en estructuras internas de memoria para permitir su procesamiento posterior (ordenamiento y representación gráfica).

El archivo se abre en modo solo lectura (O_RDONLY) y se transfiere al buffer principal `inv_buf` mediante la syscall `read`. Si ocurre un error al abrir o leer el archivo, el programa se detiene con los códigos de salida definidos para cada caso.

El algoritmo de parseo sigue una estructura secuencial y lineal:

- Salto de espacios y caracteres de control (CR, LF, tabs): Se avanza en el buffer hasta encontrar contenido válido.
- Extracción del nombre: Se recopilan los caracteres hasta encontrar el delimitador “:”. Antes de almacenarlo, se eliminan los espacios sobrantes al final del nombre.
- Almacenamiento del nombre: Se guarda en el arreglo de punteros `inv_name_ptrs`, junto con su longitud (`inv_name_lens`).
- Extracción de la cantidad: Se leen los caracteres posteriores al “:” descartando espacios. Se convierte el valor leído en un número entero acumulando dígitos.
- Almacenamiento de la cantidad: Se guarda en el arreglo `inv_qtys`. Además, se incrementa el contador global `inv_count`.

El procedimiento se repite hasta consumir la totalidad del buffer. En caso de encontrarse líneas sin el carácter “:”, estas son descartadas para evitar errores de interpretación.

Finalmente, se muestra en consola un mensaje de confirmación junto con el número total de ítems cargados, validando que el inventario fue leído correctamente.

- *Paso 3*

En esta etapa se implementó un proceso de ordenamiento de los elementos previamente cargados en memoria desde el archivo *inventario.txt*. El objetivo principal fue reordenar los ítems de acuerdo con su nombre en orden alfabético ascendente, de manera que la presentación de los resultados sea clara y consistente.

El algoritmo utilizado corresponde a un método de ordenamiento burbuja (Bubble Sort), cuya complejidad temporal es $O(n^2)$, suficiente para la escala reducida del problema (máximo 128 ítems).

La comparación entre cadenas de texto se realizó mediante la instrucción `repe_cmpsb`, que efectúa una comparación carácter por carácter hasta encontrar una diferencia o alcanzar la longitud mínima entre las dos cadenas. En caso de que

los nombres compartan un prefijo idéntico, el criterio de desempate consiste en dar prioridad al ítem con menor longitud de nombre.

Cuando se detecta que un par de elementos está desordenado, se efectúa un intercambio en bloque de la información asociada al ítem, lo cual incluye:

- El puntero al nombre (`inv_name_ptr`)
- La longitud del nombre (`inv_name_len`)
- La cantidad disponible (`inv_qty`)

Como resultado de esta fase, el inventario queda almacenado en memoria de forma ordenada alfabéticamente por nombre, lo que permite que en el siguiente paso (visualización gráfica en consola) los datos se presenten en un formato estructurado y legible.

- *Paso 4*

Dibujo del gráfico:

El objetivo de esta última etapa fue representar gráficamente el inventario en la consola mediante barras horizontales coloreadas con secuencias ANSI. Esta visualización permite observar de forma clara y comparativa las cantidades de cada ítem cargado y ordenado previamente.

- Determinación del valor máximo:

Se recorre el arreglo de cantidades (`inv_qty`) para identificar el valor más alto (`inv_max_qty`).

Este valor se utiliza como referencia para escalar proporcionalmente la longitud de cada barra.

- Iteración por ítems ordenados:

Para cada ítem, se imprime primero el nombre, seguido de un corchete de apertura (`[`), que enmarca la barra.

- Aplicación de colores ANSI:

Se emiten secuencias ANSI para aplicar el color de fondo (`color_fondo`) y el color de la barra (`color_barra`), definidos previamente en `config.ini`.

Estos valores se convierten a texto decimal y se insertan en la secuencia de escape para modificar la salida en consola.

- Escalado de la barra:

Si `inv_max_qty` es mayor que cero, la longitud de la barra se calcula como:

$$ancho = \frac{cantidad * MAX_BAR_WIDTH}{inv_max_qty}$$

donde `MAX_BAR_WIDTH` limita la barra a un máximo de 50 caracteres.

Se imprime el carácter configurado (`caracter_barra`) repetido tantas veces como indique el ancho calculado.

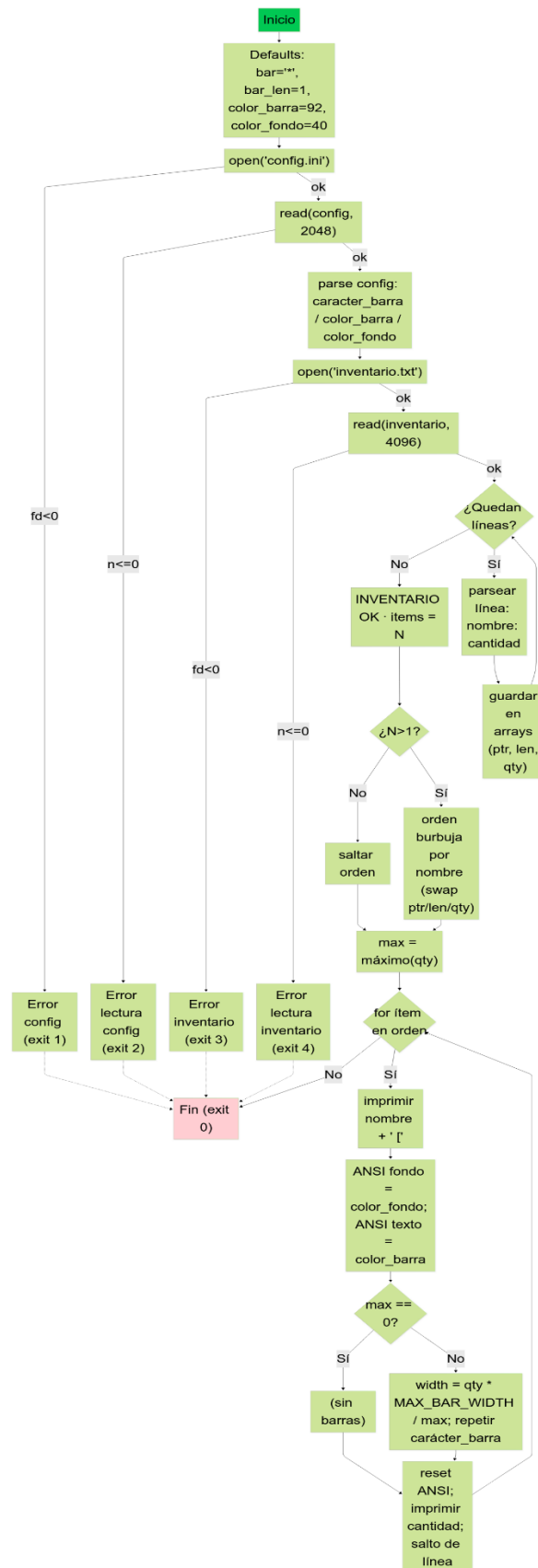
- Restablecimiento de color y finalización de la línea:

Se envía la secuencia ANSI de reset (`ESC [ 0 m`) para devolver la consola a sus colores por defecto.

Se imprime un espacio y el valor numérico de la cantidad, seguido de un salto de línea.

Así el programa concluye ofreciendo una visualización estructurada, ordenada y personalizable del inventario.

Diagrama de flujo



- Probando extensibilidad en config e inventario:

```
scisneros@DESKTOP-SANTIAGO29:~/Proyecto1/Proyecto1---Arquitectura_Computadores---S2---2025/src$ make
nasm -f elf64 -g -F dwarf -o final.o final.asm
ld -m elf_x86_64 -o final final.o
./final
caracter_barra: '&'
color_barra: 93
color_fondo: 40
INVENTARIO OK
items=22
arándano: [##### 23
banano: [##### 8
carambola: [##### 21
frambuesa: [##### 9
fresa: [##### 8
granadilla: [##### 3
guayaba: [##### 8
jocote: [##### 18
kiwis: [##### 5
mamon: [##### 4
mandarina: [##### 10
mango: [##### 4
manzanas: [##### 12
mora: [##### 7
mota: [##### 5
naranjas: [##### 25
peras: [##### 8
pitahaya: [##### 11
piñas: [##### 14
queso: [##### 27
tomate: [##### 20
uva: [##### 7
```

Conclusión

El programa integra, en ensamblador x86-64, E/S de archivos, parseo textual, ordenamiento y renderizado ANSI con manejo de errores. Su diseño modular y configurable permite ajustar fácilmente la apariencia (carácter, colores, ancho) y escalar la solución con nuevas claves o formatos de salida.